FADENA
FACULTAD DE DEFENSA NACIONAL

Universidad de la Defensa Nacional

Facultad de Defensa Nacional

Licenciatura en Ciberdefensa

Sistemas de Tratamiento de Datos

Otras tendencias y tecnologías actuales

Año: 2025

FADENA
FACULTAD DE DEFENSA NACIONAL

Universidad de la Defensa Nacional

Contenido

Integración de datos y ETL (Extract, Transform, Load) 4
Introducción 4
Fases del proceso ETL 4
Extract (Extracción) 4
Transform (Transformación) 5
Load (Carga) 6
Arquitectura general del proceso ETL 7
Herramientas y tecnologías comunes 7
Big Data 8
Contexto y definición 8
Las 5 V del Big Data 9
Arquitectura general de un sistema Big Data 9
MapReduce: paradigma de procesamiento distribuido 10
Fase Map 10
Fase Shuffle 10
Fase Reduce 10
Ventajas: 10
Limitaciones: 10
Apache Spark: evolución del procesamiento distribuido 11
Ventajas de Spark 11
Módulos principales 11
Bases de datos distribuidas 12
Conceptos básicos 12
Ejemplo de bases distribuidas populares: 12
Ecosistema moderno de Big Data 13
Ciencia de Datos y el rol de los sistemas de datos 14
Etapas de un proyecto de Ciencia de Datos 14
Recolección 14
Limpieza y preparación 14
Análisis exploratorio (EDA) 15
Modelado predictivo / descriptivo 15
Evaluación y métricas 15
Implementación y comunicación 15

El rol de los sistemas de datos 16
A. Fuentes 16
B. Integración 16
C. Almacenamiento 16
D. Procesamiento 17
E. Explotación y análisis 17
Roles dentro del ecosistema 17

Integración de datos y ETL (Extract, Transform, Load)

Introducción

En entornos organizacionales modernos, los datos se encuentran dispersos, heterogéneos y desactualizados. Pueden provenir de múltiples sistemas: bases transaccionales (OLTP), hojas de cálculo, APIs, logs de servidores, sensores IoT o plataformas externas (por ejemplo, redes sociales o marketplaces).
Para que la información pueda analizarse de forma unificada y confiable, se requiere un proceso sistemático de integración de datos, cuyo corazón es el proceso ETL.
El ETL (Extract, Transform, Load) constituye el puente entre los sistemas operacionales y los sistemas analíticos, garantizando la coherencia, calidad y trazabilidad de los datos.

Fases del proceso ETL

Extract (Extracción)

Consiste en obtener los datos desde las fuentes originales. Puede implicar:

- Conectarse a bases relacionales (MySQL, PostgreSQL, Oracle) o no relacionales (MongoDB, Redis, Elasticsearch).
- Consumir datos a través de APIs REST / SOAP.

- Leer archivos estructurados o semiestructurados (CSV, JSON, XML, Parquet).
- Ingestar flujos en tiempo real (streaming) desde Kafka o sistemas IoT.

Transform (Transformación)

Etapa crítica donde se limpian, depuran y estandarizan los datos para hacerlos consistentes.

Incluye:

- Eliminación de duplicados o valores nulos.
- Conversión de formatos (fechas, monedas, codificaciones).
- Homologación de unidades (por ejemplo, USD <-> ARS, °F <-> °C).
- Enriquecimiento con información adicional (por ejemplo, geolocalización o categorización).
- Validación de integridad referencial.
- Aplicación de reglas de negocio (por ejemplo, definir qué es “cliente activo”).

Esta fase suele involucrar reglas lógicas y de negocio, y puede realizarse mediante scripts Python, SQL, o herramientas dedicadas como Pentaho o Talend.

En entornos modernos, la transformación puede ejecutarse antes o después de la carga, dando origen a dos enfoques:

- ETL (Extract → Transform → Load) → transformación previa a la carga, típico de Data Warehouses clásicos.

- ELT (Extract → Load → Transform) → primero se carga todo en el destino (por ejemplo, BigQuery, Snowflake o Redshift) y luego se transforman los datos usando su poder de cómputo interno.

Load (Carga)

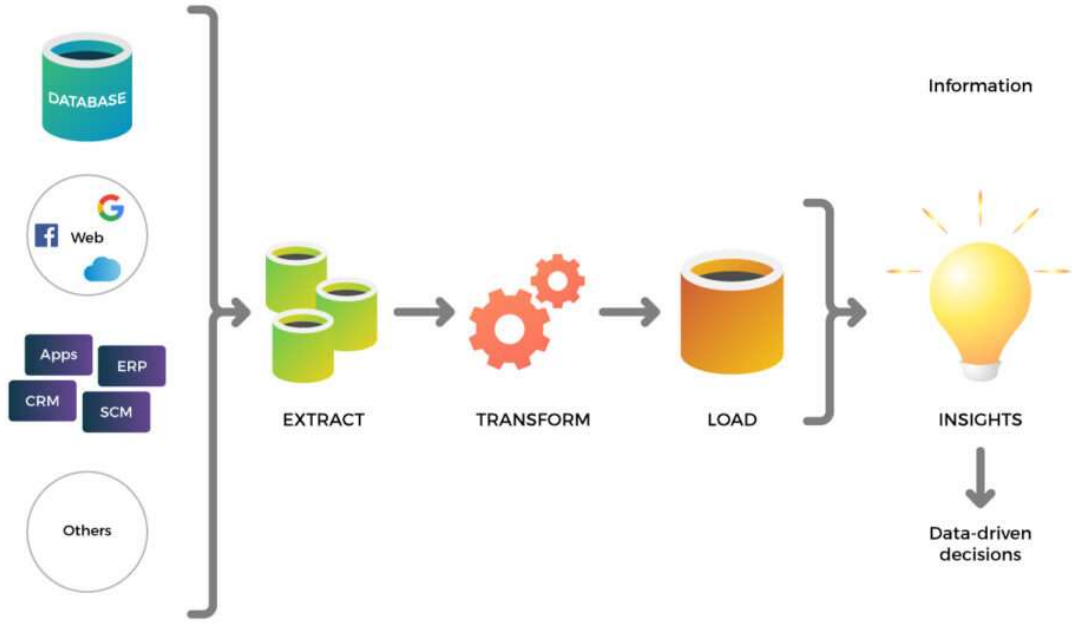
Finalmente, los datos transformados se cargan en un repositorio destino para su análisis o explotación.
Los destinos típicos incluyen:

- Data Warehouse (DW): almacén analítico estructurado en esquemas estrella o copo de nieve.
- Data Lake: repositorio masivo que almacena datos en bruto, estructurados y no estructurados.
- Data Mart: subconjunto temático del DW (por ejemplo, ventas o logística).
- Bases analíticas en la nube (Snowflake, BigQuery, Redshift, etc.).

La carga puede ser:

- Completa: se reemplaza toda la tabla o dataset.
- Incremental: solo se insertan los nuevos registros o actualizaciones (mejor performance).
- Por lotes (batch) o en streaming (tiempo real).

Arquitectura general del proceso ETL



Herramientas y tecnologías comunes

Tipo	Ejemplos	Características
ETL tradicional (on-premise)	Pentaho Data Integration (Kettle), Talend, Informatica PowerCenter	Interfaz visual, programación de flujos, manejo batch.
Orquestadores modernos	Apache Airflow, Luigi, Prefect	Definición de pipelines en Python, gestión de dependencias y scheduling.
ETL ligero con Python	pandas, sqlalchemy, pyodbc, requests	Ideal para proyectos pequeños o automatizaciones.
Cloud ETL / ELT	AWS Glue, Azure Data Factory, Google Dataflow	Integración nativa con servicios cloud y escalabilidad automática.

Big Data

Contexto y definición

El término Big Data no se refiere únicamente al tamaño de los datos, sino a la complejidad del procesamiento y del valor que puede extraerse de ellos.

A medida que las organizaciones comenzaron a generar terabytes y petabytes de datos heterogéneos (transacciones, sensores, redes sociales, imágenes, logs, etc.), los modelos tradicionales (bases relacionales, servidores únicos) dejaron de ser suficientes.

Big Data surge entonces como un conjunto de tecnologías y metodologías para:

- Almacenar grandes volúmenes de datos en forma distribuida.
- Procesarlos en paralelo en múltiples nodos.
- Analizarlos en tiempo real o batch.
- Extraer valor útil para la toma de decisiones o modelos predictivos.

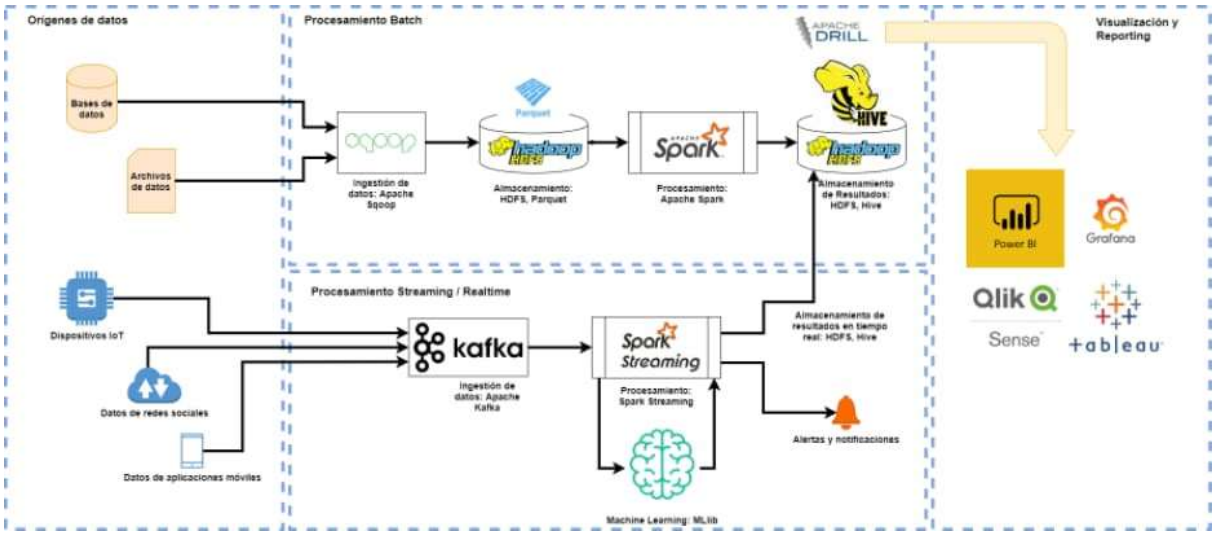
FADENA
FACULTAD DE DEFENSA NACIONAL

Universidad de la Defensa Nacional

Las 5 V del Big Data

V	Significado	Descripción	Ejemplo
Volumen	Cantidad	Datos masivos: desde GB a PB o EB.	Logs de servidores, imágenes satelitales.
Velocidad	Ritmo de generación y actualización	Streaming continuo o lotes rápidos.	Tweets por segundo, sensores IoT.
Variedad	Diversidad de formatos y fuentes	Estructurados, semiestructurados, no estructurados.	Tablas SQL, JSON, video, texto.
Veracidad	Calidad y confiabilidad	Datos incompletos o contradictorios requieren validación.	Noticias falsas, registros duplicados.
Valor	Utilidad analítica	El propósito final: información para decidir.	Modelos de riesgo, predicción de demanda.

Arquitectura general de un sistema Big Data



MapReduce: paradigma de procesamiento distribuido

MapReduce fue introducido por Google (2004) y adoptado en Hadoop como modelo de cómputo paralelo.

Fase Map

Divide el trabajo en tareas independientes, cada una ejecutada cerca de los datos.

Ejemplo: contar palabras en millones de documentos.

Cada “mapper” genera pares clave-valor (<palabra, 1>).

Fase Shuffle

El sistema agrupa todas las claves iguales (todas las instancias de una misma palabra).

Fase Reduce

Combina los valores asociados a cada clave y genera el resultado final (<palabra, total>).

Ventajas:

- Escalabilidad horizontal (agregar nodos = más rendimiento).
- Tolerancia a fallos.
- Ideal para procesamiento batch (logs, analítica histórica).

Limitaciones:

- Alto overhead por escritura/lectura en disco.
- No apto para cargas interactivas o tiempo real.

Apache Spark: evolución del procesamiento distribuido

Apache Spark es la evolución de Hadoop/MapReduce.

Procesa datos en memoria (in-memory), evitando la escritura en disco tras cada operación, y permite un flujo de datos continuo.

Ventajas de Spark

Característica	Descripción
Velocidad	10–100× más rápido que MapReduce.
API unificada	Soporta SQL, streaming, ML y gráficos en un mismo framework.
Lenguajes	Compatible con Python (PySpark), Scala, Java y R.
In-memory	Guarda resultados intermedios en RAM.
Escalabilidad	Corre en clusters propios o en la nube (EMR, Dataproc, Databricks).

Módulos principales

- Spark Core: motor base de ejecución distribuida.
- Spark SQL: consultas SQL y optimización de datos estructurados (DataFrames).
- Spark Streaming: procesamiento en tiempo real.
- MLlib: machine learning distribuido.
- GraphX: análisis de redes y grafos.

Bases de datos distribuidas

En el ecosistema Big Data, los datos no se guardan en un único servidor, sino fragmentados (shards) y replicados entre varios nodos.

Conceptos básicos

Término	Descripción
Sharding	División horizontal de la tabla en múltiples nodos.
Replication	Copias idénticas de los datos para alta disponibilidad.
Consistency	Garantía de coherencia entre réplicas.
Partition tolerance	Sistema sigue operando aunque falle un nodo o red.
CAP theorem	No se puede tener simultáneamente Consistencia, Disponibilidad y Tolerancia a Particiones; solo dos de tres.

Ejemplo de bases distribuidas populares:

Tecnología	Tipo	Características
HBase	Columnar (sobre HDFS)	Inspirada en Google BigTable. Alta velocidad de lectura/escritura.
Cassandra	NoSQL, columnar	Arquitectura peer-to-peer, alta disponibilidad (AP).
MongoDB Sharded Cluster	Documental	Shards distribuidos por rango o hash. Escalabilidad horizontal.
Elasticsearch	Indexación / búsqueda distribuida	Basado en shards y réplicas. Ideal para logs y analítica textual.

FADENA
FACULTAD DE DEFENSA NACIONAL

Universidad de la Defensa Nacional

Ecosistema moderno de Big Data

Capa	Herramientas destacadas	Descripción
Ingesta / ETL	Kafka, NiFi, Airflow	Movimiento y orquestación de datos.
Almacenamiento	HDFS, S3, Delta Lake, Iceberg	Repositorios escalables y distribuidos.
Procesamiento batch	Spark, Flink, Hadoop	Cómputo paralelo sobre grandes volúmenes.
Procesamiento streaming	Kafka Streams, Spark Streaming	Datos en tiempo real.
Consulta interactiva	Hive, Presto, Dremio, BigQuery	SQL distribuido sobre datos masivos.
Analítica avanzada	Spark MLlib, TensorFlow, PyTorch	Machine Learning y predicciones.
Visualización	Kibana, Superset, Power BI	Dashboards sobre índices o data lakes.

Ciencia de Datos y el rol de los sistemas de datos

La Ciencia de Datos (Data Science) es una disciplina interdisciplinaria que combina:

- Estadística y Matemática: para modelar y entender los datos.
- Programación: para automatizar procesos y construir algoritmos.
- Conocimiento del dominio: para contextualizar y convertir hallazgos en decisiones.

Su objetivo es extraer conocimiento, generar predicciones y facilitar decisiones basadas en evidencia.
"Data Science es el puente entre los datos crudos y la toma de decisiones inteligentes."

Etapas de un proyecto de Ciencia de Datos

Un flujo típico de trabajo se conoce como ciclo de vida de la Ciencia de Datos o pipeline analítico:

Recolección

Obtención de datos desde distintas fuentes:

- Bases relacionales (SQL)
- NoSQL (MongoDB, Redis, Elasticsearch)
- APIs (Twitter, ACLED, MercadoLibre, etc.)
- Sensores, logs o web scraping

Limpieza y preparación

Normalización, eliminación de valores nulos, outliers y duplicados. En esta etapa se decide la calidad y usabilidad de los datos.

Herramientas: pandas, OpenRefine, SQL, Airflow.

Análisis exploratorio (EDA)

- Identificar patrones, correlaciones, distribuciones.
- Visualizar relaciones entre variables.

Herramientas: matplotlib, seaborn, Power BI, Kibana.

Modelado predictivo / descriptivo

- Aplicar algoritmos de Machine Learning (regresiones, árboles, clustering, redes neuronales).

Librerías: scikit-learn, XGBoost, TensorFlow

Evaluación y métricas

Validar resultados según el tipo de problema:

- Clasificación → accuracy, precision, recall, F1
- Regresión → RMSE, MAE, R^2
- Clustering → silhouette score, inertia

Implementación y comunicación

- Poner modelos en producción (APIs, Dashboards, servicios).
- Comunicar resultados con storytelling visual.

Herramientas: Flask, FastAPI, Streamlit, Dash, Grafana, Power BI.

El rol de los sistemas de datos

Los sistemas de datos constituyen la infraestructura invisible pero fundamental que soporta todo proyecto de Ciencia de Datos. Sin una arquitectura de datos sólida, el análisis se vuelve frágil, inconsistente o directamente inviable.

A. Fuentes

Los datos pueden provenir de:

- Sistemas transaccionales (OLTP): ERP, CRM, bases SQL.
- Sensores IoT, logs, redes sociales, APIs.
- Fuentes abiertas (open data, ACLED, NASA, etc.).

B. Integración

A través de pipelines ETL o ELT:

- Limpieza, normalización, enriquecimiento.
- Almacenamiento en Data Warehouse o Data Lake.
- Herramientas: Airflow, Kafka, Pentaho, Spark.

C. Almacenamiento

Dependerá del tipo y volumen de datos:

Tipo de dato	Sistema sugerido	Ejemplo
Estructurado	SQL relacional	MySQL, PostgreSQL
Semi estructurado	NoSQL documental	MongoDB
No estructurado	Data Lake	Hadoop HDFS, S3
Indexado / Texto	Búsqueda distribuida	Elasticsearch

D. Procesamiento

- Batch: grandes volúmenes, no urgentes (Spark, Hive, Flink).
- Streaming: datos en tiempo real (Kafka, Spark Streaming).

E. Explotación y análisis

- Business Intelligence: dashboards, reporting.
- Machine Learning: predicción y clasificación.
- Ciencia de Datos Avanzada: modelado estadístico, IA, NLP, visión artificial.

Roles dentro del ecosistema

Rol	Enfoque	Herramientas clave
Data Engineer	Construye y mantiene pipelines y bases de datos.	SQL, Spark, Airflow, ETL, Docker
Data Analyst	Explora y visualiza datos para decisiones tácticas.	SQL, Power BI, Tableau, Excel
Data Scientist	Diseña modelos predictivos y experimentos.	Python, scikit-learn, TensorFlow
ML Engineer	Implementa y optimiza modelos en producción.	MLflow, FastAPI, CI/CD
Data Architect	Diseña la infraestructura global de datos.	AWS, Azure, GCP, Hadoop